Assignment no 3

Name :

Maryam Riaz

Roll no:

Fa23-BCS-296

Subject :

Advance database

Submitted to:

Sir Anwaar Mehdi

Insert at least 5,000 dummy records into the collection

```
universitydb> const departments = ["CS", "SE", "EE", "ME", "CE"];
              const cities = ["Lahore", "Karachi", "Islamabad", "Peshawar", "Quetta"];
              const names = ["Ali", "Sara", "Hamza", "Zara", "Usman", "Ayesha",
                             "Bilal", "Hina", "Kashif", "Sana"];

              let batch = [];
              for (let i = 1; i <= 5000; i++) {
                batch.push({
                  name:       names[i % names.length] + " " + i,
                  rollNo:     "FA23-BCS-" + String(i).padStart(3, "0"),
                  department: departments[i % departments.length],
                  semester:   (i % 8) + 1,
                  cgpa:       parseFloat((Math.random() * 3 + 1).toFixed(2)),
                  city:       cities[i % cities.length]
                });
                if (batch.length === 500) {
                  db.students.insertMany(batch);
                  batch = [];
                }
              }
              if (batch.length > 0) db.students.insertMany(batch);

              print("Total inserted:", db.students.countDocuments());
```

1. Runaquery to find students by rollNo and analyze its performance using: .explain("executionStats")

2. Create an appropriate index for the query.

```
> db.students.createIndex({ rollNo: 1 });
< rollNo_1
```

3. Runthe same query again and compare: a. Execution time b. Documents examined

```
> db.students.find({ rollNo: "FA23-BCS-001" }
              .explain("executionStats");
< {
    explainVersion: '1',
    queryPlanner: {
      namespace: 'universitydb.students',
      parsedQuery: {
        rollNo: {
          '$eq': 'FA23-BCS-001'
        }
      },
      indexFilterSet: false,
      queryHash: '0DB48064',
      planCacheShapeHash: '0DB48064',
      planCacheKey: 'FEC0A1B5',
      optimizationTimeMillis: 9,
      maxIndexedOrSolutionsReached: false,
      maxIndexedAndSolutionsReached: false,
      maxScansToExplodeReached: false,
      prunedSimilarIndexes: false,
      winningPlan: {
```

Without index
Stage: **COLLSCAN**
Docs examined: ~5,000
Time: ~15–25 ms

With index
Stage: **IXSCAN → FETCH**
Docs examined: 1
Time: <1 ms

4. Create a compound index on department and cgpa.

```
> db.students.createIndex({ department: 1, cgpa: -1 });
< department_1_cgpa_-1
```

Execute a query that finds all students from the "CS" department and sorts them by cgpa in descending order

```
>_MONGOSH
> db.students.find({ department: "CS" })
             .sort({ cgpa: -1 })
             .explain("executionStats");
< {
    explainVersion: '1',
    queryPlanner: {
      namespace: 'universitydb.students',
      parsedQuery: {
        department: {
          '$eq': 'CS'
        }
      },
      indexFilterSet: false,
      queryHash: '34856902',
      planCacheShapeHash: '34856902',
      planCacheKey: '9D8FCFD3',
      optimizationTimeMillis: 3,
      maxIndexedOrSolutionsReached: false,
      maxIndexedAndSolutionsReached: false,
      maxScansToExplodeReached: false,
      prunedSimilarIndexes: false,
      winningPlan: {
        isCached: false,
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: {
            department: 1,
            cgpa: -1
          },
          indexName: 'department_1_cgpa_-1',
          isMultiKey: false,
```

Display all indexes created on the collection

```
> db.students.getIndexes();
< [
    { v: 2, key: { _id: 1 }, name: '_id_' },
    { v: 2, key: { rollNo: 1 }, name: 'rollNo_1' },
    {
      v: 2,
      key: { department: 1, cgpa: -1 },
      name: 'department_1_cgpa_-1'
    }
  ]
```

Write a short conclusion explaining how indexes improved query performance in MongoDB

Indexes significantly improved query performance in MongoDB by reducing the time required to search and retrieve data. Instead of scanning every document in a collection, MongoDB used indexes to quickly locate the required records, resulting in faster execution of queries, better efficiency, and improved overall database performance, especially for large datasets.